

Agente DQN para
jugar Super
Mario Bros {

<Por="Francisco Antonio López
Ricardez"/>

}



Contexto y objetivo {

que Mario aprenda una política para avanzar el nivel usando solo píxeles y recompensas.

¿Qué es lo que se busca?

Entrenar un agente de Reinforcement Learning para jugar Super Mario Bros automáticamente.

¿Qué algoritmo se utiliza?

Deep Q-Network (DQN).

}

Arquitectura {

Train/Test

scripts
principales que
crean el entorno,
instancian al
agente y ejecutan
los episodios

agent

la clase Mario, que
implementa el agente
DQN; memoria de replay,
política ϵ -greedy,
actualización de Q,
guardado de
checkpoints.

wrappers

Optimizar el
rendimiento
general.
(redimensionar,
saltar
frames)

metrics

registro de
métricas y
generación de
gráficas de
entrenamiento.

neural

La red
convoluciona
l que estima
los valores
Q

}

Entorno y preprocesamiento

Entorno {

Entorno super mario

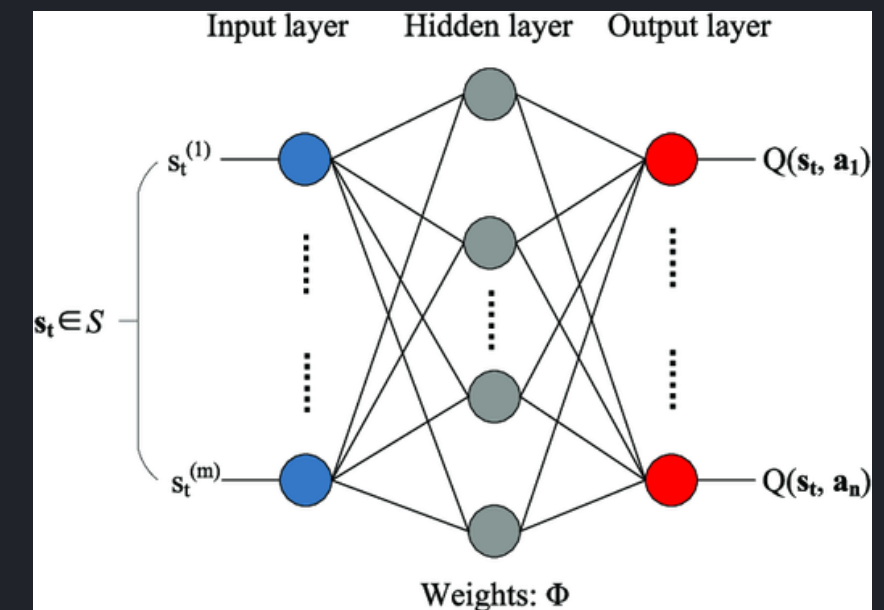
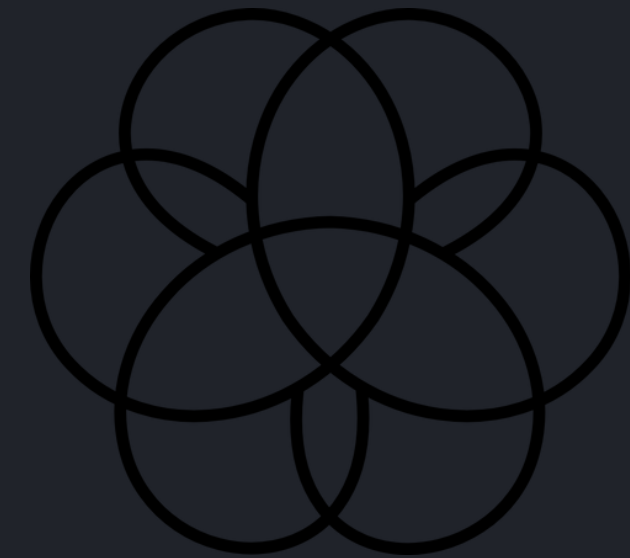
Se usa:

- gym_super_mario_bros
- nes_py

para crear el entorno.

¿Qué algoritmo se utiliza?

Deep Q-Network (DQN).



}

Preprocesamiento de observaciones {



SKIPFRAME

repite la misma acción varios frames y suma la recompensa



GRAYSCALEOBSERVATION

pasar a escala de grises



RESIZEOBSERVATION(84X84)

reducir tamaño a 84×84 para simplificar la entrada a la red



TRANSFORMOBSERVATION

normalización a [0,1]



FRAMESTACK

pilar 4 frames para dar información de movimiento

}

Red Neuronal

```
# Red principal (Q-online)
self.online = nn.Sequential(
    nn.Conv2d(channels, 32, kernel_size=8, stride=4),
    nn.ReLU(),

    nn.Conv2d(32, 64, kernel_size=4, stride=2),
    nn.ReLU(),

    nn.Conv2d(64, 64, kernel_size=3, stride=1),
    nn.ReLU(),

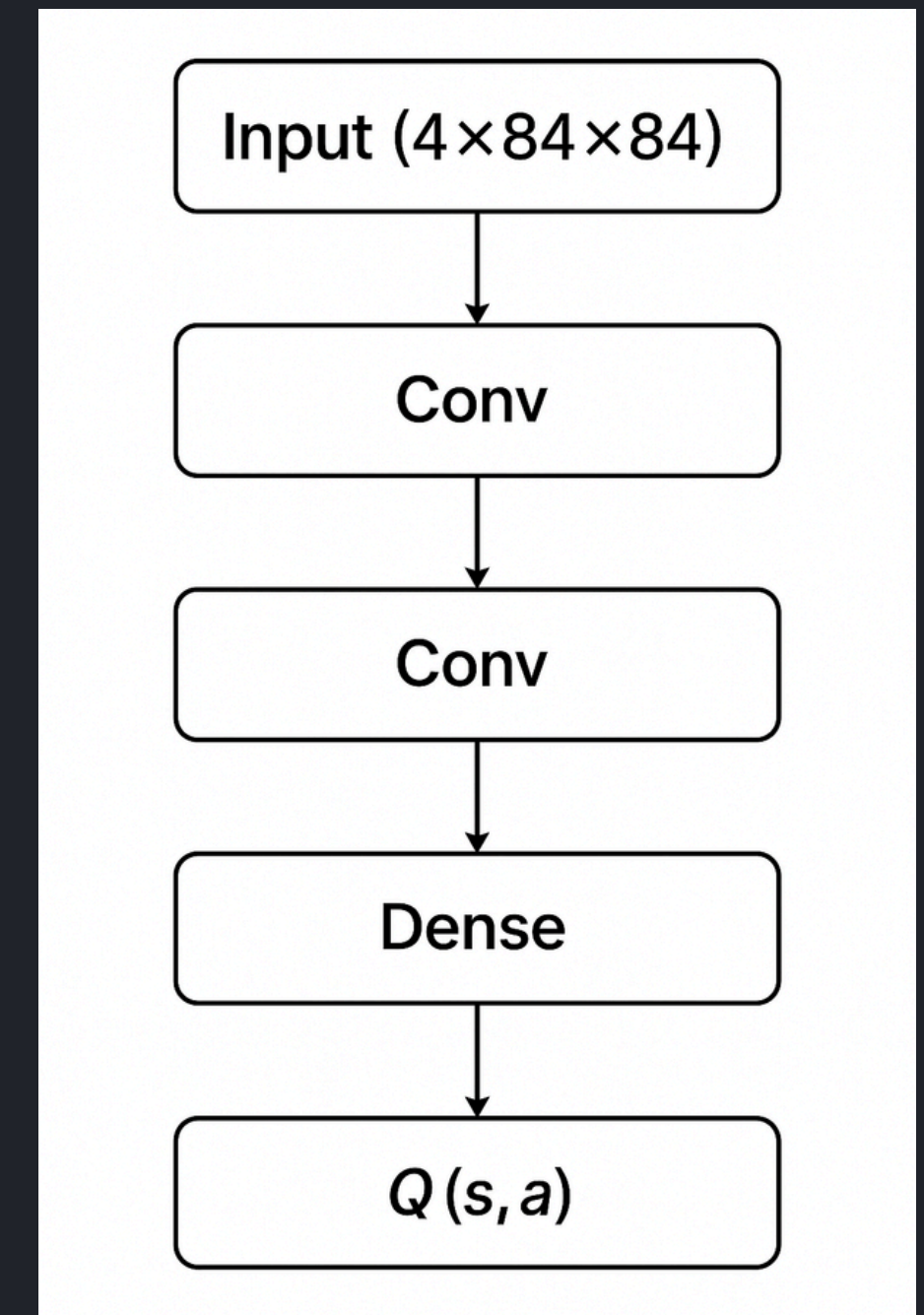
    nn.Flatten(),
    nn.Linear(3136, 512),
    nn.ReLU(),
    nn.Linear(512, output_dim)
)

# Red objetivo (Q-target)
self.target = copy.deepcopy(self.online)
```

Arquitectura:

- 3 capas Conv2d con ReLU.
- Flatten.
- Dos capas lineales, la última con tamaño igual al número de acciones.

Se implementa como red online y se copia a una red target congelada para estabilizar el entrenamiento.



Agente DQN



MEMORIA DE REPLAY

- Se usa un deque para guardar experiencias con tamaño máximo 100,000.
agent

POLÍTICA ϵ -GREEDY

- Con probabilidad ϵ elige una acción aleatoria.
- Con probabilidad $1-\epsilon$ elige la acción con mayor Q usando

APRENDIZAJE, UPDATE DE Q

- recall
- td_estimate
- td_target
- learn

}

Bucle de entrenamiento

```
for episode in range(num_episodes):
    obs = env.reset()

    while True:
        # Acción del agente
        action = agent.act(obs)

        # Avanzar en el entorno
        next_obs, reward, terminated, info = env.step(action)

        # Guardar transición
        agent.cache(obs, next_obs, action, reward, terminated)

        # Aprendizaje
        q_val, loss_val = agent.learn()

        # Registrar métricas del paso
        logger.log_step(reward, loss_val, q_val)

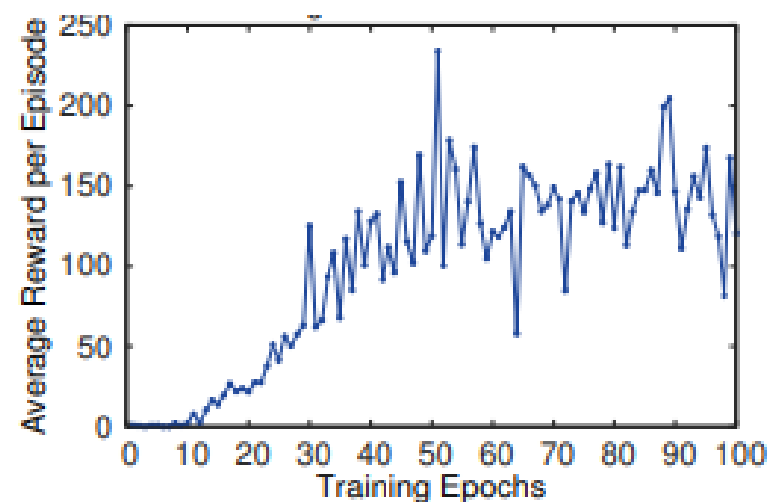
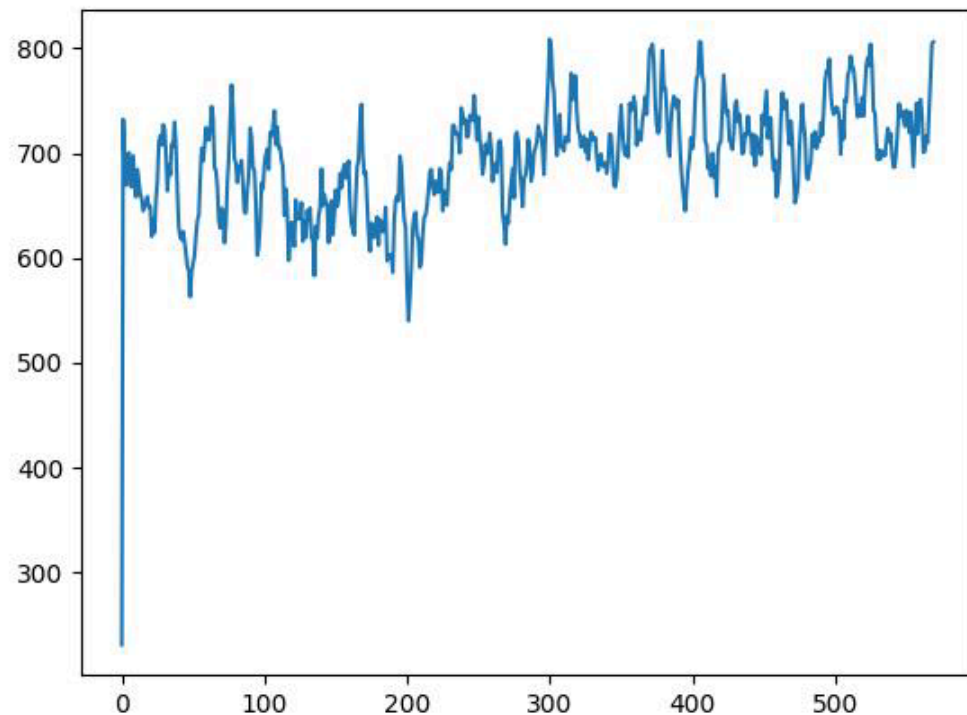
        # Avanzar estado
        obs = next_obs

        # Fin de episodio
        if terminated or info.get("flag_get"):
            break
```

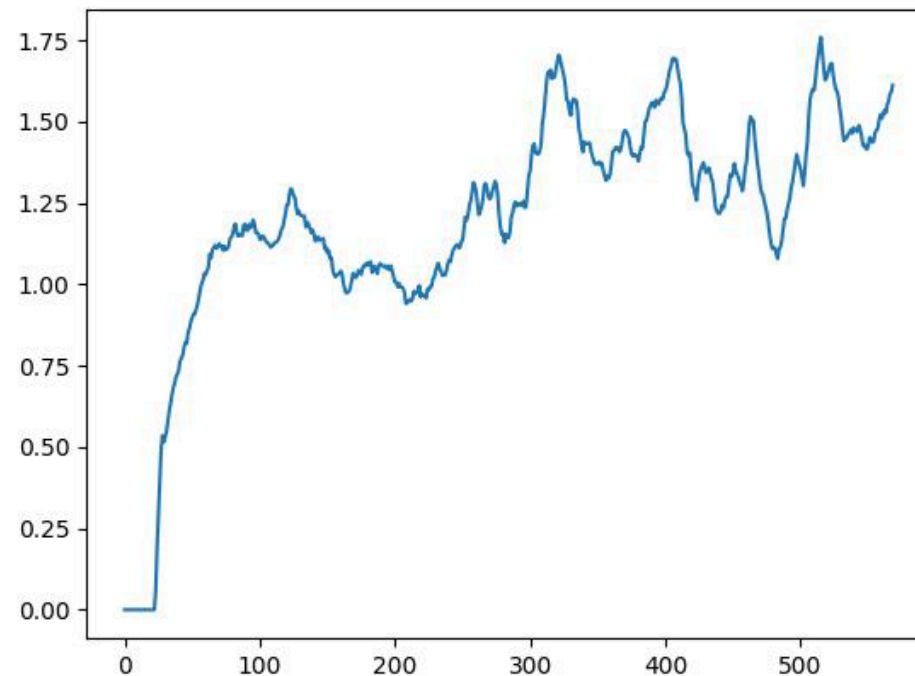
Registro de métricas y gráficas

Metrics que se guardan {

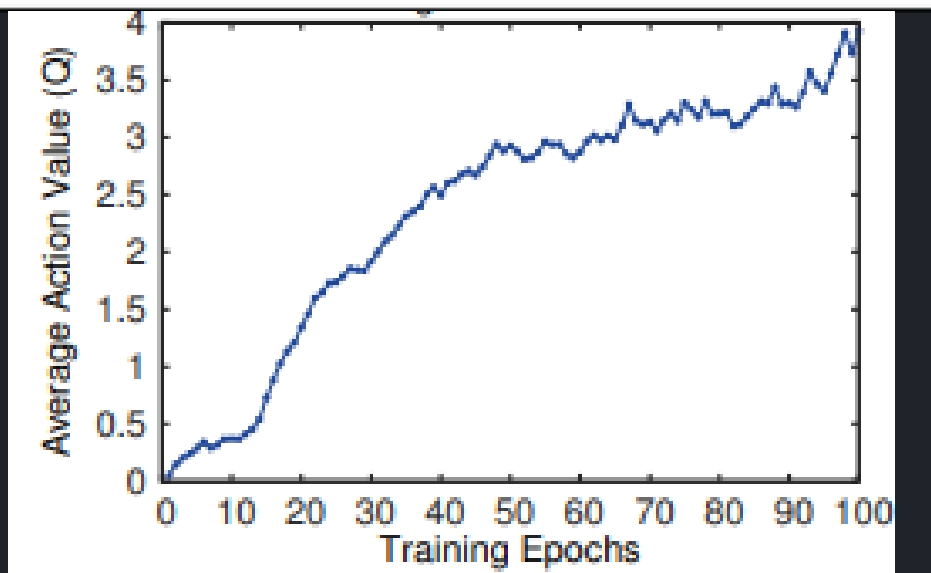
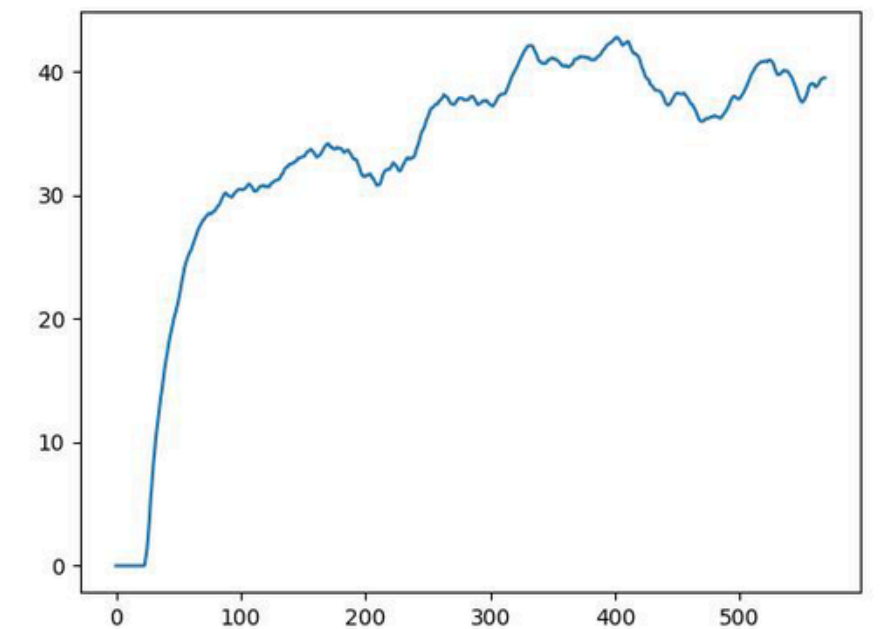
Recompensa
por episodio



Pérdida
promedio.



Valor Q
promedio



Test

```
env = create_env()
env.reset()

run_dir = Path("checkpoints") / datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
run_dir.mkdir(parents=True, exist_ok=True)

checkpoint_path = Path("trained_mario.chkpt")

agent = Mario(
    state_dim=(4, 84, 84),
    action_dim=env.action_space.n,
    save_dir=run_dir,
    checkpoint=checkpoint_path,
    test=True
)
```


Limitaciones y áreas de mejora

Aspectos de mejora {

Limitaciones

- Tiempo de entrenamiento muy largo.
- Solo se usan dos acciones.

Posibles mejoras

- Entrenar más episodios.
- Ajustar recompensas.
- Ampliar el espacio de acciones.

}